

TÜR İPUÇLARI

Python dinamik tipli bir dildir. Yani bir değişkenin veri tipi **çalışma zamanında** (runtime) belirlenir. Ancak büyük projelerde bu durum hatalara sebep olabilir. İşte bu noktada **tür ipuçları** (type hints) devreye girer. Tür ipuçları Python 3.5 ile tanıtılmış ve sonraki sürümlerde geliştirilmiştir.

Tür İpuçlarının Temelleri

Tür ipuçları, değişkenlere, fonksiyon parametrelerine ve dönüş değerlerine beklenen veri tipini belirtmemizi sağlar. Bu sadece bir ipucudur; Python bu tipleri çalışma zamanında kontrol etmez. Ancak mypy gibi araçlar bu ipuçlarını kullanarak kod analizini yapar.

```
def toplan(a: int, b: int) -> int:
    return a + b
sonuc: int = toplan(5, 3)
print(sonuc)
# Çıktı: 8
```

Yukarıdaki örnekte a ve b parametrelerinin int tipinde olması gerektiği, fonksiyonun int döndüreceği ve sonuc değişkeninin int olduğu belirtilmiştir.

Temel Veri Tipleri

```
isim: str = "İlhan"
yas: int = 35
boy: float = 1.75
aktif: bool = True
sayilar: list = [1, 2, 3]
koordinat: tuple = (10, 20)
ayarlar: dict = {"dil": "tr"}
```

Gelişmiş Tür İpuçları

Daha karmaşık tür ipuçları için typing modülü kullanılır:

```
from typing import List, Dict, Optional, Union, Tuple

# Liste, elemanları int
sayilar: List[int] = [1, 2, 3, 4, 5]

# Sözlük, anahtarları str, değerleri int
puanlar: Dict[str, int] = {"Ali": 85, "Ayşe": 92}

# None olabilir
def bul_ogrenci(id: int) -> Optional[str]:
    if id == 1:
        return "Ahmet"
    return None

# Birden fazla tip kabul eder (Python 3.10+ için | kullanılabilir)
def isle(veri: Union[int, str]) -> str:
    return str(veri)

# Python 3.10+ ile daha basit sözdizimi
def isle_yeni(veri: int | str) -> str:
    return str(veri)
```

Python 3.10+ Modern Özellikler

Eşleşme Desenleri

Python 3.10 ile gelen en önemli özelliklerden biri **eşleşme desenleridir** (pattern matching). `match-case` ifadesi ile karmaşık koşullu ifadeleri daha okunabilir hale getirebiliriz:

```
def http_durumu(kod: int) -> str:
    match kod:
        case 200:
            return "Başarılı"
        case 404:
            return "Bulunamadı"
        case 500:
            return "Sunucu Hatası"
        case _:
            return "Bilinmeyen Durum"

print(http_durumu(200))
# Çıktı: Başarılı
```

Daha karmaşık eşleşmeler de yapılabilir:

```
def komut_isle(komut: tuple) -> str:
    match komut:
        case ["git", yön]:
            return f"{yön} yönüne gidiliyor"
        case ["al", nesne, miktar]:
            return f"{miktar} adet {nesne} alınıyor"
        case ["dur"]:
            return "Duruldu"
        case _:
            return "Bilinmeyen komut"

print(komut_isle(["git", "kuzey"]))
# Çıktı: kuzey yönüne gidiliyor
print(komut_isle(["al", "elma", 5]))
# Çıktı: 5 adet elma alınıyor
```

Union İşleci

Python 3.10'dan önce birden fazla tipi belirtmek için Union kullanıyorduk. Artık daha basit `|` işlecini kullanabiliriz:

```
# Eski yöntem (hala geçerli)
from typing import Union
def islem(sayi: Union[int, float]) -> Union[int, float]:
    return sayi * 2

# Yeni yöntem (Python 3.10+)
def islem_yeni(sayi: int | float) -> int | float:
    return sayi * 2

# None için de kullanılabilir
def bul(anahtar: str) -> str | None:
    sozluk = {"a": "değer"}
    return sozluk.get(anahtar)
```

Statik Tip Kontrolü

Python kodundaki tür ipuçlarını kontrol eden bir araç `mypy`'dir. Kodunuzu çalıştırmadan önce tip hatalarını bulmanızı sağlar. Kurulumu `pip install mypy` komutu ile yapılır.

Örnek bir hatalı kod:

```
# ornek.py
# def toplam(a: int, b: int) -> int:
#     return a + b

# sonuc: int = toplam(5, "10") # Hata: str kullanıldı
```

mypy ile kontrol:

```
$ mypy ornek.py
ornek.py:4: error: Argument 2 to "toplam" has incompatible type "str"; expected "int"
```

Generic Tipler ve TypeVar

Generic tipler, fonksiyonların farklı tiplerle çalışmasını sağlar:

```
from typing import TypeVar, List

T = TypeVar('T') # Generic tip değişkeni

def ilk_eleman(liste: List[T]) -> T | None:
    return liste[0] if liste else None

sayilar = [1, 2, 3]
print(ilk_eleman(sayilar)) # mypy int olarak anlar
# Çıktı: 1

kelimeler = ["merhaba", "dünya"]
print(ilk_eleman(kelimeler)) # mypy str olarak anlar
# Çıktı: merhaba
```

Literal ve Final

Literal, bir değişkenin sadece belirli değerleri alabileceğini belirtir:

```
from typing import Literal

def yon_degistir(yon: Literal["kuzey", "güney", "doğu", "batı"]) -> str:
    return f"{yon} yönüne dönülüyor"

print(yon_degistir("kuzey")) # Geçerli
# Çıktı: kuzey yönüne dönülüyor
# print(yon_degistir("yukarı")) # mypy hata verir
```

Final, bir değişkenin değiştirilmemesi gerektiğini belirtir:

```
from typing import Final

MAX_BAGLANTI: Final[int] = 100
# MAX_BAGLANTI = 200 # mypy hata verir
```

Protocol ve Structural Subtyping

Protocol, bir sınıfın hangi yöntemlere sahip olması gerektiğini belirtir. Kalıtım gerektirmez:

```
from typing import Protocol

class Cizilebilir(Protocol):
    def ciz(self) -> None:
        ...

class Daire:
    def ciz(self) -> None:
```

```

        print("Daire çiziliyor")

class Kare:
    def ciz(self) -> None:
        print("Kare çiziliyor")

def sekil_ciz(sekil: Cizilebilir) -> None:
    sekil.ciz()

sekil_ciz(Daire()) # Geçerli - ciz() metodu var
# Çıktı: Daire çiziliyor
sekil_ciz(Kare()) # Geçerli - ciz() metodu var
# Çıktı: Kare çiziliyor

```

TypedDict

TypedDict, **sözlüklerin** (dict) yapısını tanımlamak için kullanılır:

```

from typing import TypedDict

class Ogrenci(TypedDict):
    isim: str
    yas: int
    sinif: str

ogrenci: Ogrenci = {
    "isim": "Ahmet",
    "yas": 15,
    "sinif": "9-A"
}

def ogrenci_bilgisi(o: Ogrenci) -> str:
    return f"{o['isim']}, {o['yas']} yaşında, {o['sinif']} sınıfında"

print(ogrenci_bilgisi(ogrenci))
# Çıktı: Ahmet, 15 yaşında, 9-A sınıfında

```

Callable

Fonksiyonları parametre olarak alan fonksiyonlar için **callable** kullanılır:

```

from typing import Callable

def uygula(func: Callable[[int, int], int], a: int, b: int) -> int:
    return func(a, b)

def topla(x: int, y: int) -> int:
    return x + y

def carp(x: int, y: int) -> int:
    return x * y

print(uygula(topla, 5, 3))
# Çıktı: 8
print(uygula(carp, 5, 3))
# Çıktı: 15

```

Özet ve En İyi Uygulamalar

- Tür ipuçları opsiyoneldir ancak büyük projelerde çok yararlıdır
- **mypy** ile kodunuzu düzenli olarak kontrol edin
- Python 3.10+ kullanıyorsanız **|** operatörünü tercih edin

- Generic tipler kod tekrarını önler
- Protocol, duck typing'i tip güvenli hale getirir
- Tüm fonksiyonlarda dönüş tipini belirtin
- Public API'lerde tür ipuçları kullanmak özellikle önemlidir
- IDE'ler (PyCharm, VS Code) tür ipuçlarını kullanarak daha iyi otomatik tamamlama sağlar
- Karmaşık veri yapılarında `TypedDict` kullanın
- Değiştirilmemesi gereken değerler için `Final` kullanın